Defining and Solving Reinforcement Learning Environments

A 5 by 4 Grid Navigation Task with Deterministic and Stochastic Transitions

Minuu Lakshmi Shanmugam
B01157604
CS 4/565 Introduction to AI, Fall 2026
Due Oct 26, 2026 at 11:59 PM

Abstract

This report defines a small grid navigation task as a Markov decision process, implements it in Gymnasium form, and solves it with Q learning and with SARSA. The deterministic version always carries out the requested action. The stochastic version carries out the requested action with probability 0.70 and slips to another action with probability 0.30. Q learning reaches the optimal total of 10.5 on the deterministic task and 9.5 on a greedy stochastic run. SARSA reaches the same deterministic optimum. Tuning shows that epsilon decay has the largest effect on learning speed in this task, and the recommended setting is given in Section 6.

1 Environment definition

The task is navigation on a 5 by 4 grid. The agent starts at (0, 0), the top left cell, and must reach (4, 3), the bottom right cell. Three special cells give extra reward. The episode ends at the goal, or after 100 steps. The 100 step limit is a time limit, not a task failure, and it is reported as truncation.

An observation is a position plus a record of which special cells have already been collected. Position alone is not enough as a state. If position alone were the state, a second visit to the same cell would look identical to the first visit, even though the reward on the second visit is different. The collected record removes that ambiguity and keeps the task Markovian.

There are 20 positions and 8 possible collected records, so the observation space has 160 states. The state number is position index times 8 plus the collected mask, where position index is row times 4 plus column, and mask bits follow the order (0, 3), (2, 3), (3, 2). The action space has 4 actions: 0 down, 1 up, 2 right, 3 left. A move into a wall leaves the agent in place, with the ordinary step cost.

Rewards

Event	Reward	When it is paid
Ordinary step	-0.1	Every step, except goal and first special visit
Special cell (0, 3)	1.0	First visit only, in each episode
Special cell (2, 3)	2.0	First visit only, in each episode
Special cell (3, 2)	3.0	First visit only, in each episode
Goal cell (4, 3)	5.0	On reaching the goal, episode ends

There are five distinct reward values, which meets the requirement of at least four. The best possible deterministic episode collects all three specials and then reaches the goal in 9 steps. The total is 1.0 plus 2.0 plus 3.0 plus 5.0, minus 0.5 for the five ordinary steps, which is 10.5. No deterministic episode can earn more, because each special can be paid only once and the goal ends the episode.

Transition rules

In the deterministic version, the requested action always occurs, so the transition probability for that outcome is 1. In the stochastic version, the requested action occurs with probability 0.70, and each of the other three actions occurs with probability 0.10. The four probabilities sum to 1.00. The stochastic version keeps the same grid, start, goal, specials, actions, and reward meanings as the deterministic version. Only the transition probabilities change.

The implementation uses the Gymnasium methods __init__, reset, step, and render [4, 5]. Reset returns an observation and an information dictionary. Step returns an observation, a reward, a termination flag, a truncation flag, and an information dictionary. Both environment classes pass gymnasium.utils.env_checker.check_env, which checks that observations and actions belong to their spaces and that reset and step return values in the expected form.

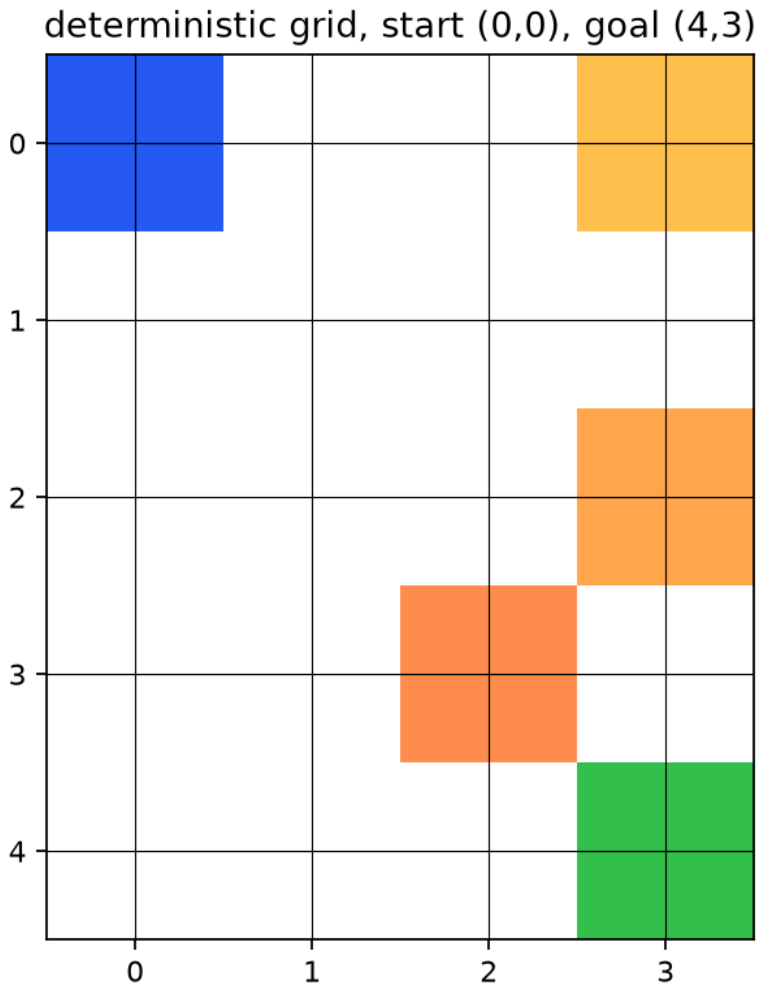


Figure 1: Deterministic grid. Start is the top left cell. Goal is the bottom right cell. Special cells are marked in the interior.

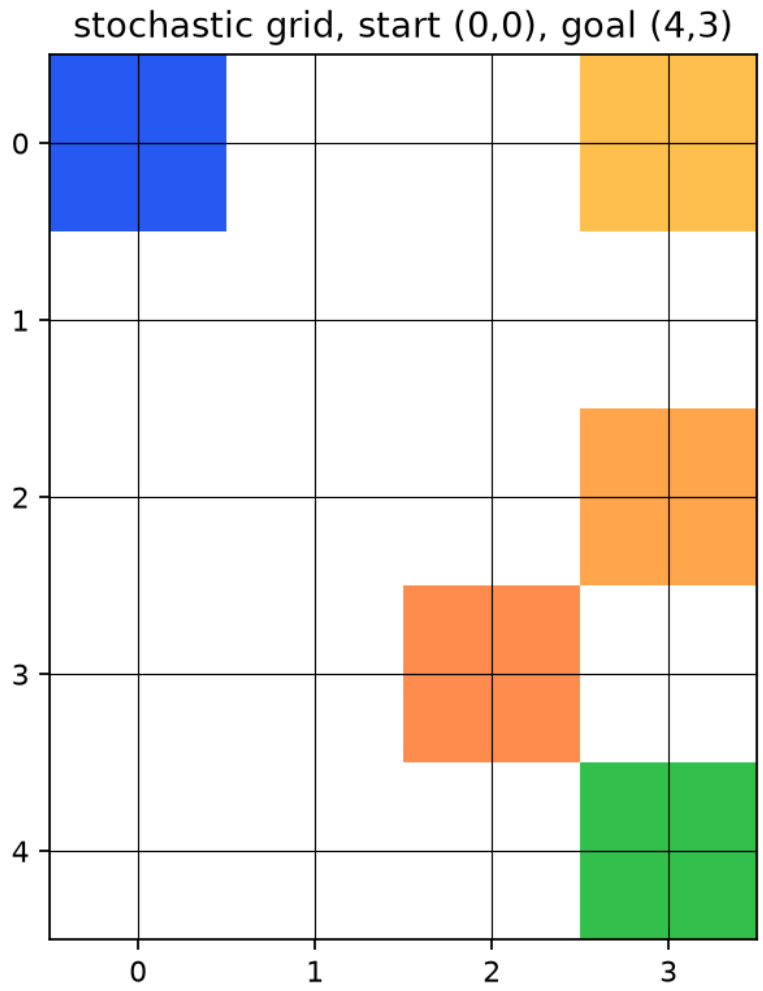


Figure 2: Stochastic grid. The layout, start, goal, and special cells are identical to the deterministic grid. Only the transition probabilities differ.

2 Random agent validation

A random agent was run for 40 timesteps in each environment, with seed 2026. The full traces are in Appendix A. Neither random run reaches the goal, which is expected for random play on this grid. The deterministic run earns a total of negative 0.8, with positive rewards of 2.0 at timestep 31 and 1.0 at timestep 35. The stochastic run earns a total of negative 2.9, with a positive reward of 1.0 at timestep 27. The traces show wall moves staying in place, specials paying only on first visit, and the stochastic run slipping off the requested path. This confirms that the environment logic works before any learning is applied.

Stochastic transition check

A 40 step trace shows that the stochastic environment runs, but it does not by itself show that the transition frequencies match the intended probabilities. To check frequencies directly, the agent was placed at (1, 1) on every trial, so all four actual actions lead to distinct positions, with no wall clipping and no special cell. The requested action was right on every trial, for 1000 trials, with seed 2026.

Actual action	Expected frequency	Observed count	Observed frequency
Down	0.10	106	0.106
Up	0.10	90	0.090
Right, requested	0.70	712	0.712
Left	0.10	92	0.092

Table 2: Stochastic transition frequencies, 1000 trials, start (1, 1), requested action right. Observed frequencies are close to the intended probabilities, with the small differences expected from sampling variation. This check supports the transition design in Section 1, in addition to the random traces.

3 Q learning

Q learning is an off policy temporal difference method [1, 2]. The action value table starts at zero. At each step, the agent acts epsilon greedy: with probability epsilon it picks a random action, otherwise it picks an action with the highest current table value, breaking ties at random. The update is table value plus learning rate times reward plus discount factor times the best next table value, minus the current table value. After termination or truncation, the future term is zero, so both methods in this report treat the time limit the same way. The 100 step limit is defined as part of this task, so an episode ending at that limit is treated as the end of a finite horizon episode for learning. This is a modeling choice for this task, not a claim about Gymnasium semantics in general. Training uses learning rate 0.3, discount factor 0.9, epsilon starting at 1.0, epsilon minimum 0.05, epsilon decay 0.99, 500 episodes, and at most 100 steps per episode, with seed 2026 and a training episode seed of seed plus episode number.

Figure 3 shows Q learning on the deterministic environment. The first 50 episode mean is 6.56, the last 50 episode mean is 10.47, the last 100 episode mean is 10.47, the overall mean is 9.85, and the final episode earns 10.5. A greedy run from the saved table, with no random actions, earns 10.5 in 9 steps and terminates at the goal. The path is (0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (2, 2), (3, 2), (4, 2), (4, 3). That is the optimal total found in Section 1.

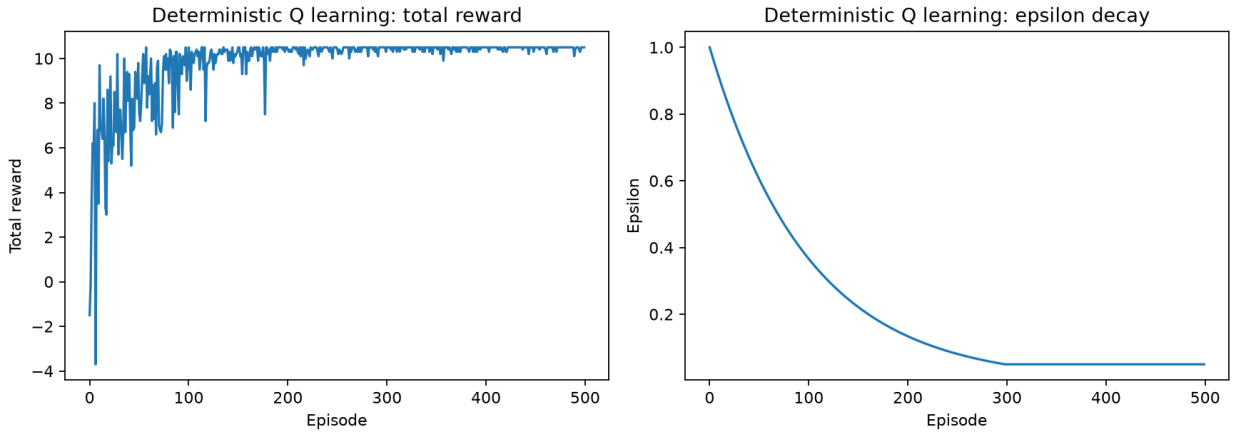


Figure 3: Q learning on the deterministic environment. Left: total reward per episode. Right: epsilon decay per episode.

Figure 4 shows Q learning on the stochastic environment, with the same base setting. The first 50 episode mean is 4.86, the last 50 episode mean is 8.89, the last 100 episode mean is 8.93, the overall mean is 8.40, and the final episode earns 8.8. A greedy run from the saved table earns 9.5 in 19 steps and terminates at the goal, after slipping off and back onto the optimal route. The stochastic curve is lower and noisier than the deterministic curve, because slips add steps and variance. Learning still converges to near optimal play, it just cannot reach 10.5 reliably when actions slip.

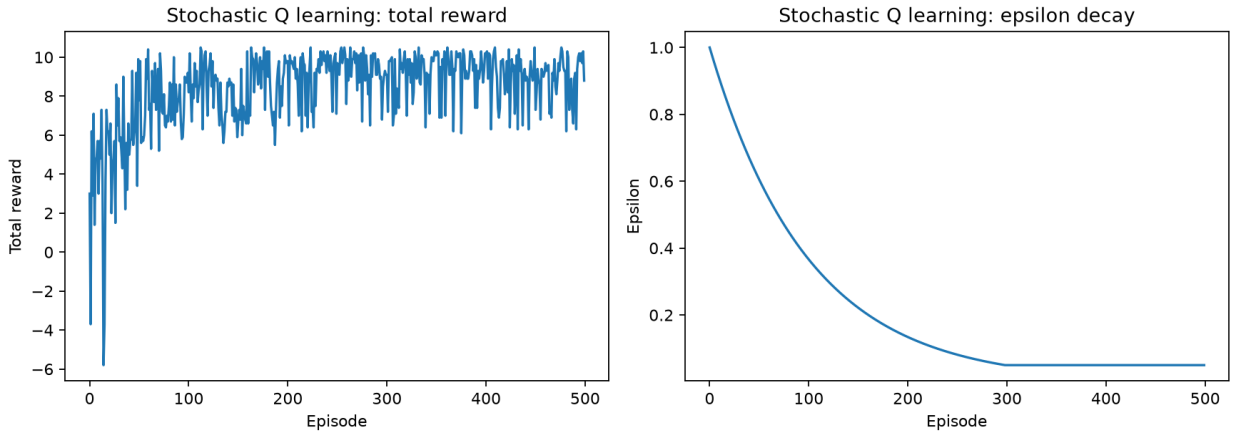


Figure 4: Q learning on the stochastic environment. Left: total reward per episode. Right: epsilon decay per episode.

4 SARSA

SARSA is an on policy temporal difference method [1, 3]. It differs from Q learning in one place in the update. Q learning uses the best next action value, whether or not that action is taken. SARSA uses the value of the action actually taken next, chosen by the same epsilon greedy policy. After termination or truncation, its target is just the observed reward, as in Q learning. The base setting is the same as for Q learning, and the test is on the deterministic environment.

Figure 5 shows SARSA on the deterministic environment. The first 50 episode mean is 5.25, the last 50 episode mean is 10.36, the last 100 episode mean is 10.40, the overall mean is 9.19, and the final episode earns 10.4. A greedy run from the saved SARSA table earns 10.5 in 9 steps and terminates at the goal. The final step goes through (3, 3) instead of (4, 2), but the total is the same optimal total. SARSA converges to the same policy as Q learning here, with slightly slower average learning, which is expected because SARSA learns from exploratory actions as well as from greedy actions.

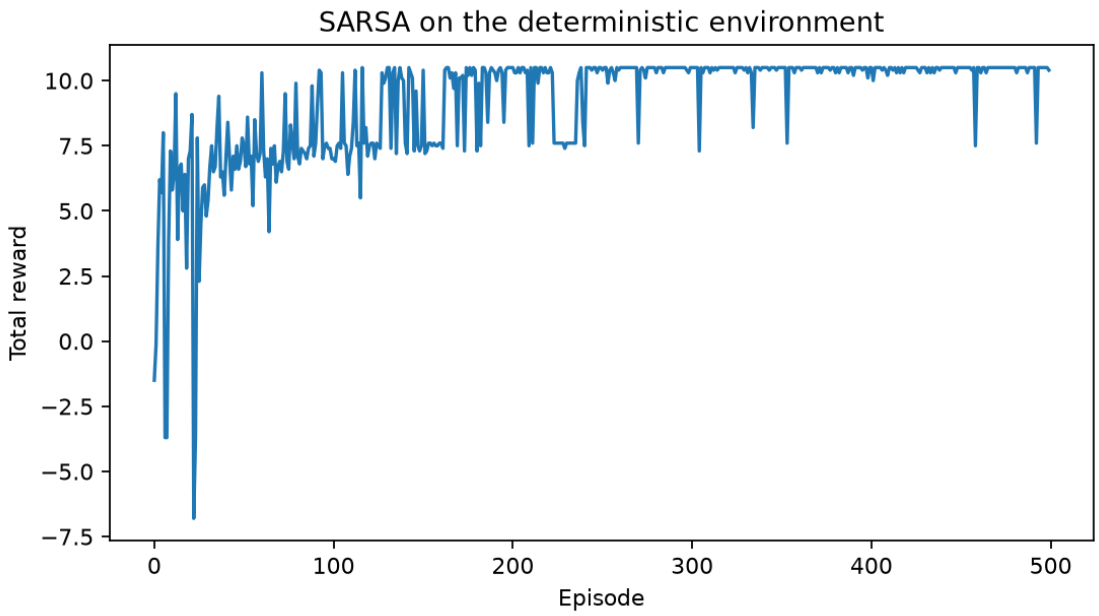


Figure 5: SARSA on the deterministic environment. Total reward per episode.

5 Hyperparameter tuning

Tuning changes one parameter at a time, with all other values at base, on the deterministic environment, with the same seed. Each parameter is tested at three values. The base setting is learning rate 0.3, discount factor 0.9, epsilon decay 0.99, epsilon minimum 0.05, epsilon start 1.0, 500 episodes, and 100 steps per episode. Table 1 gives the means for every run. Figures 6 to 9 show the 20 episode mean reward for each comparison. Figures 10 to 13 show four individual detail runs, so the tuning section contains 8 reward graphs in total. Each tuning run is a single fixed seed comparison, so the results show which setting learns fastest on this task and seed, not a claim that the same setting would win on every seed.

Parameter	Value	First 50	Last 50	Overall
Learning rate	0.1	5.98	10.47	9.18
Learning rate	0.3	6.56	10.47	9.85
Learning rate	0.7	6.08	10.45	9.85
Discount factor	0.7	6.41	10.46	9.79
Discount factor	0.9	6.56	10.47	9.85
Discount factor	0.99	6.42	10.46	9.84
Epsilon decay	0.95	8.77	10.47	10.27
Epsilon decay	0.99	6.56	10.47	9.85
Epsilon decay	0.995	3.90	10.40	9.15
Epsilon minimum	0.0	6.56	10.49	9.86
Epsilon minimum	0.05	6.56	10.47	9.85
Epsilon minimum	0.2	6.56	10.19	9.73

Table 1: Tuning results on the deterministic environment. Means are total reward per episode.

Learning rate

Learning rate 0.1 and 0.3 tie at a last 50 mean of 10.47, and 0.7 is 10.45, within 0.02 of that tie. Learning rate 0.3 and 0.7 share the highest overall mean, 9.85, and 0.1 is clearly slower early. The largest rate does not destabilise learning here, because the task is small and rewards are bounded, but the middle value is the safer choice and it is kept as base.

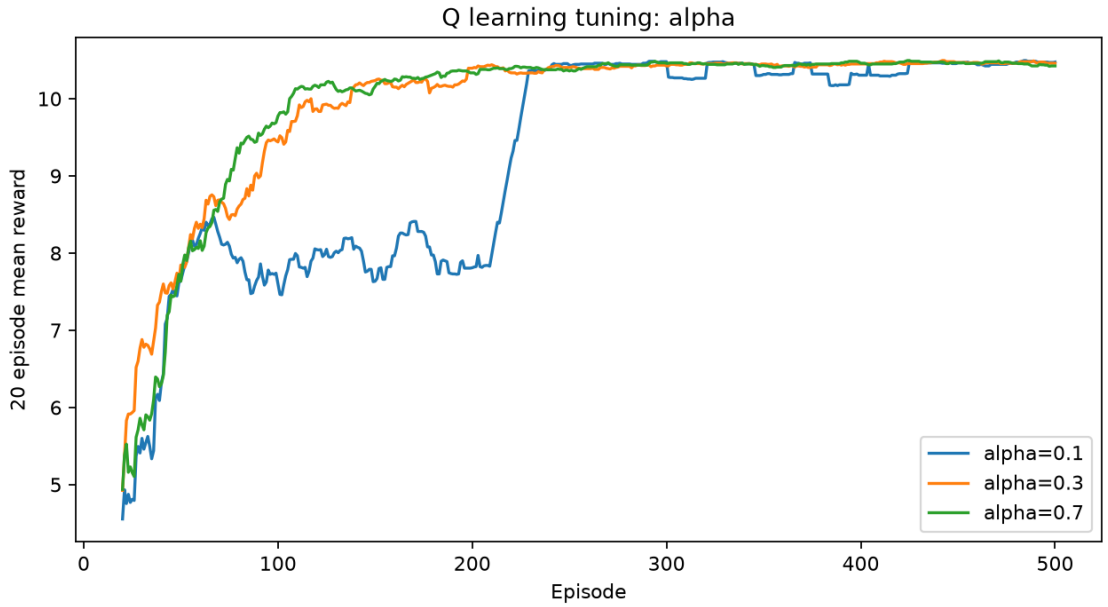


Figure 6: Tuning learning rate. Values 0.1, 0.3, and 0.7, with all other values at base. The vertical axis is 20 episode mean reward.

Discount factor

Discount factor has little effect on the final result in this task, because the optimal route is short and the goal reward dominates. All three values reach a last 50 mean of at least 10.46. The middle value 0.9 has the highest overall mean, by a small margin, and it is kept as base.

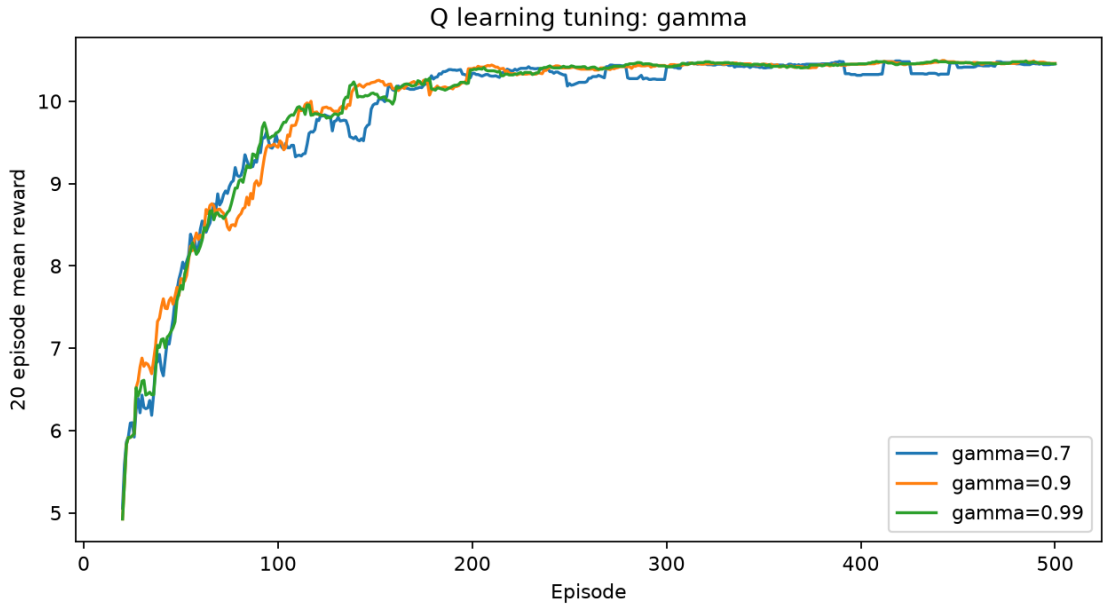


Figure 7: Tuning discount factor. Values 0.7, 0.9, and 0.99, with all other values at base. The vertical axis is 20 episode mean reward.

Epsilon decay

Epsilon decay has the largest effect on learning speed in this task. Decay 0.95 reaches a first 50 mean of 8.77, compared with 6.56 for 0.99 and 3.90 for 0.995, while all three reach a last 50 mean of at least

10.40. Faster decay means less random play early, which helps here because the task is small and the optimal route is found quickly. Decay 0.95 is therefore the most efficient value in this task, even though 0.99 was the base value.

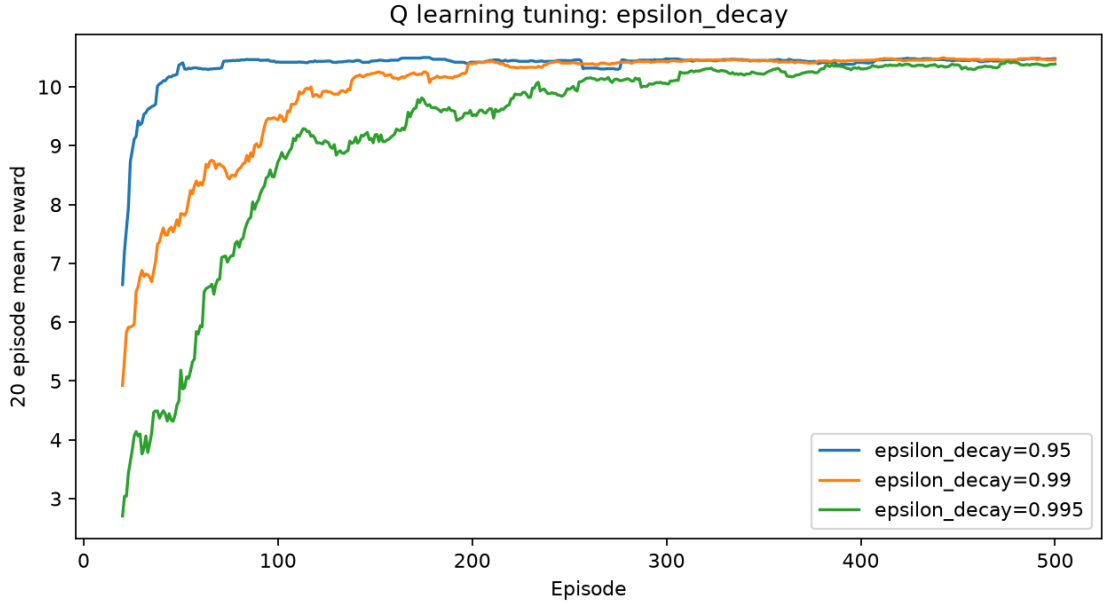


Figure 8: Tuning epsilon decay. Values 0.95, 0.99, and 0.995, with all other values at base. The vertical axis is 20 episode mean reward.

Epsilon minimum

Epsilon minimum affects only late training, because epsilon has not decayed to the floor in the first 50 episodes. That is why the first 50 means are identical across its three values. A minimum of 0.0 has a last 50 mean of 10.49, slightly above 10.47 for 0.05, but a minimum of 0.2 drops to 10.19 and its final episode is 10.2, which shows that too much late random play hurts. Minimum 0.05 is kept, because it keeps a small amount of late exploration as insurance, at a cost of only 0.02 in last 50 mean compared with 0.0, and it avoids the clear drop at 0.2.

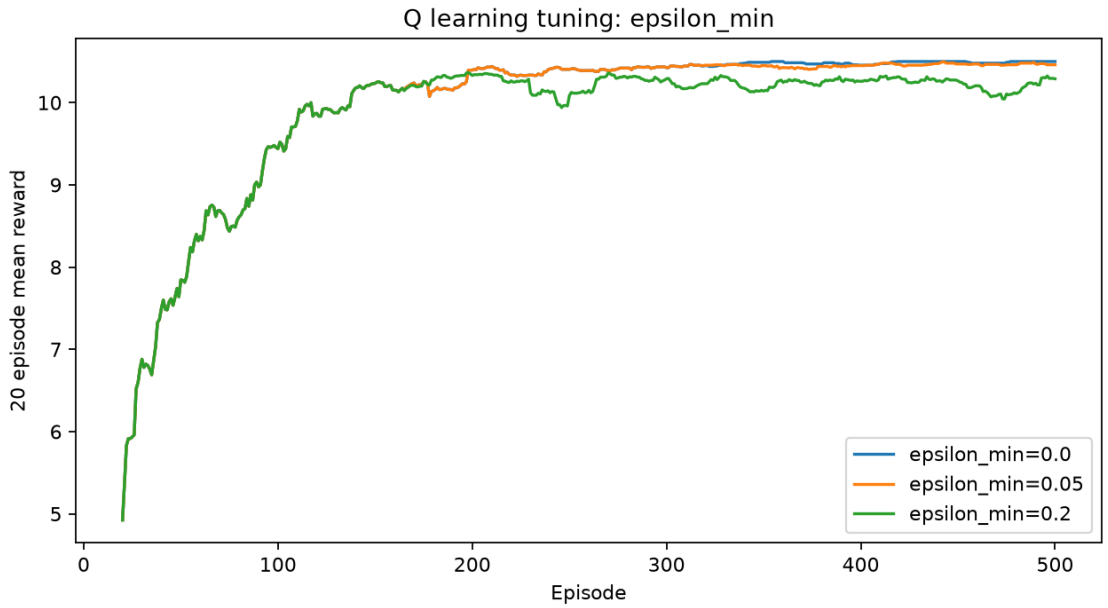


Figure 9: Tuning epsilon minimum. Values 0.0, 0.05, and 0.2, with all other values at base. The vertical axis is 20 episode mean reward.

Detail figures

Figure 10 shows learning rate 0.1 on its own. Early learning is slow, with the curve staying below the other learning rates for about 200 episodes, before reaching the same late total. This detail explains why the comparison figure for learning rate shows a clear early gap, even though all three values converge late.

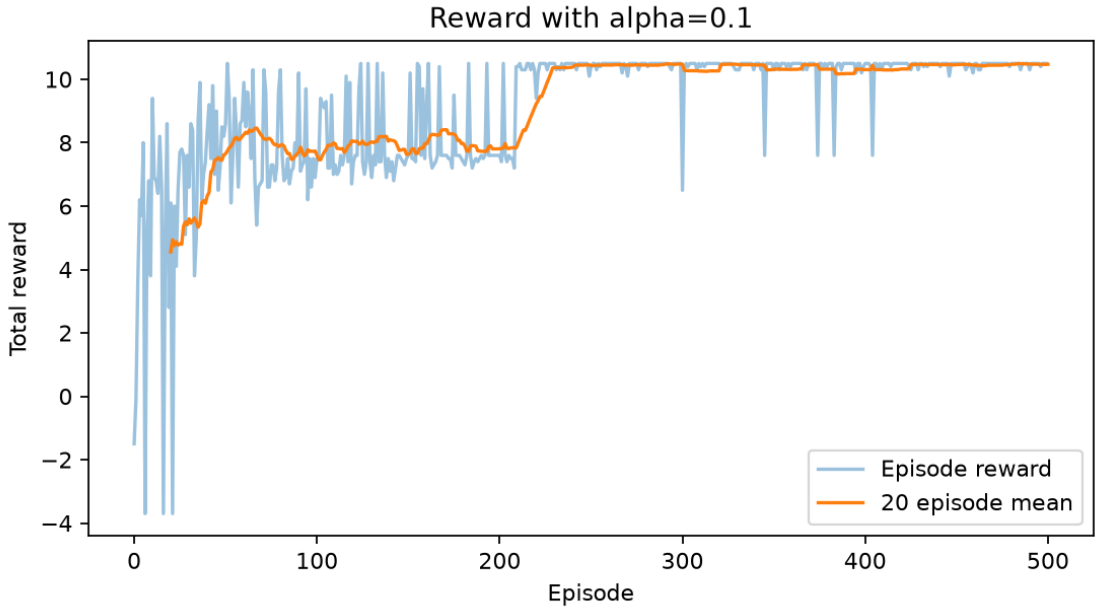


Figure 10: Detail for learning rate 0.1. Episode reward in light colour, 20 episode mean in dark colour.

Figure 11 shows epsilon decay 0.95 on its own, which is the recommended value. The curve rises fastest in the first 50 episodes, which matches its first 50 mean of 8.77, the highest in Table 1. This detail supports the recommendation, because it shows speed, not just a high final total.

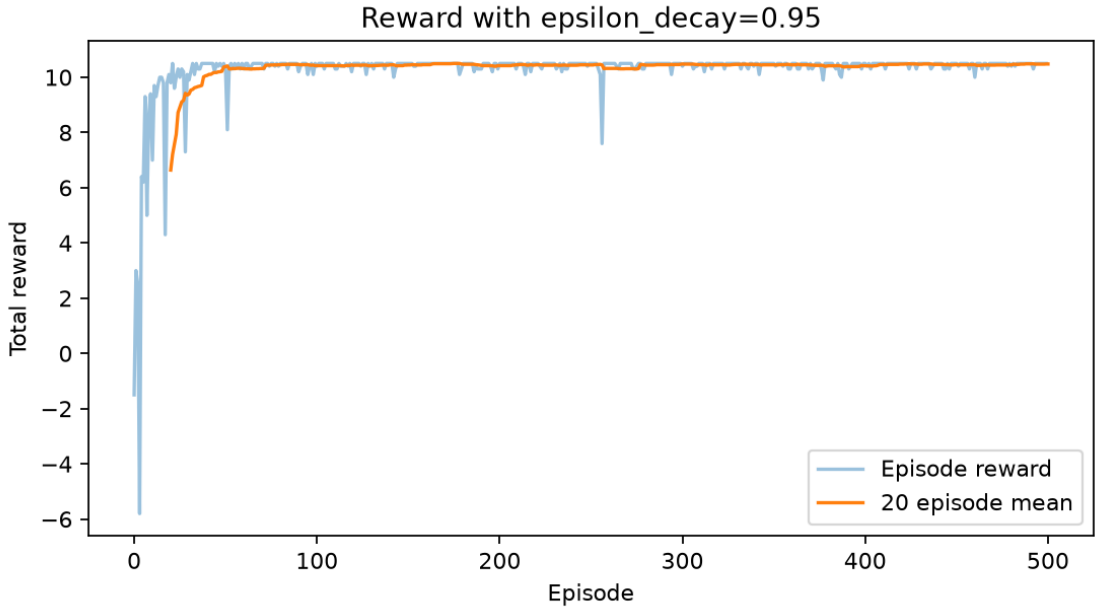


Figure 11: Detail for epsilon decay 0.95, the most efficient value in this task.

Figure 12 shows epsilon decay 0.995 on its own. Early learning is the slowest in the tuning set, with a first 50 mean of 3.90, because exploration stays high for longer. Late learning still reaches the same total, which shows that slow decay costs time, not final performance, in this task.

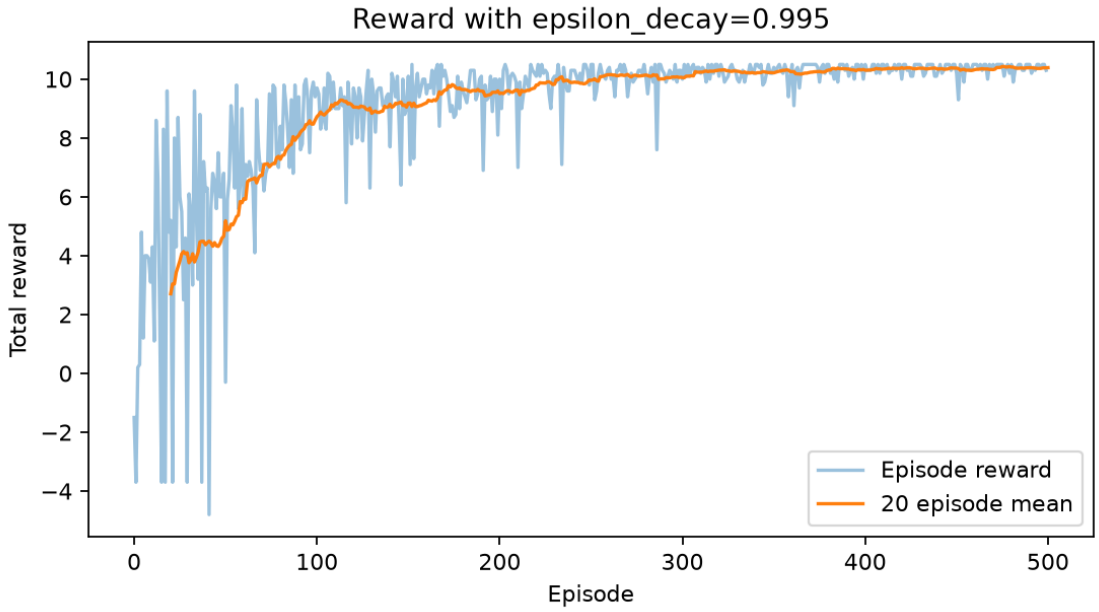


Figure 12: Detail for epsilon decay 0.995. Slowest early learning, same late convergence.

Figure 13 shows epsilon minimum 0.2 on its own. Late learning is clearly lower than for 0.0 and 0.05, with a last 50 mean of 10.19 and a final episode of 10.2. This detail explains why a high minimum is not recommended, even though it keeps exploration late, because it prevents the agent from exploiting what it has learned.

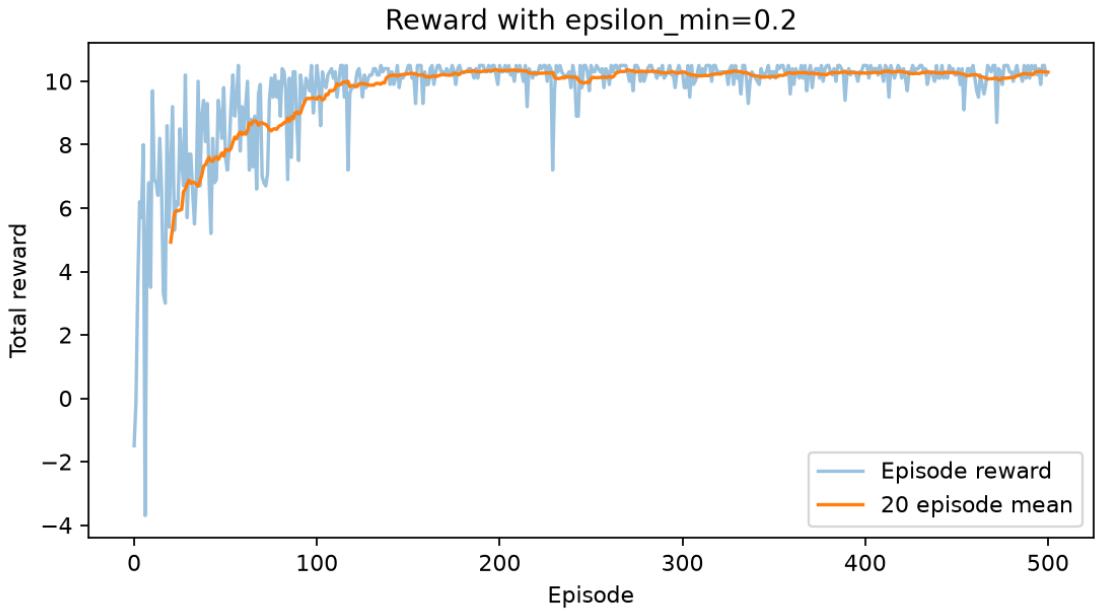


Figure 13: Detail for epsilon minimum 0.2. Late performance is clearly lower.

6 Recommended setting and conclusions

Recommended setting: learning rate 0.3, discount factor 0.9, epsilon decay 0.95, epsilon start 1.0, epsilon minimum 0.05, 500 episodes, and 100 steps per episode. This recommended setting is the epsilon decay 0.95 tuning run, with all other values at base. That run has first 50 mean 8.77, last 50 mean 10.47, and overall mean 10.27. This setting is the most efficient in the tuning runs, not just the highest final reward. It learns fastest, reaches the optimal last 50 mean, and keeps a small amount of late exploration.

The deterministic and stochastic environments are the same task with different transition uncertainty. In the deterministic version, the agent can plan an exact route and reach 10.5 reliably. In the stochastic version, the same policy slips, takes more steps, and earns less on average, even after learning. The collected mask is what makes the state representation correct in both versions, because it records which specials have already been paid. Q learning and SARSA both solve the deterministic task, and Q learning also solves the stochastic task to near optimal play.

References

[1] Sutton, R. S., and Barto, A. G. Reinforcement Learning: An Introduction. 2nd edition. MIT Press, 2018.

[2] Watkins, C. J. C. H. Learning from Delayed Rewards. PhD thesis, University of Cambridge, 1989.

[3] Rummery, G. A., and Niranjan, M. On-Line Q-Learning Using Connectionist Systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department, 1994.

[4] Towers, M., Terry, J. K., Kwiatkowski, A., et al. Gymnasium. GitHub repository, 2023. https://gymnasium.farama.org

[5] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. OpenAI Gym. arXiv:1606.01540, 2016.

Appendix A Random agent traces, 40 timesteps each

Reward is shown as a signed value. An ordinary step is -0.1. A special cell shows its positive value. Goal reward does not occur in either random trace. State and next state are encoded as position index times 8 plus collected mask, as defined in Section 1. Actions are 0 down, 1 up, 2 right, 3 left.

Deterministic trace

Step	State	Action	Next state	Reward	Done
1	0	left	0	-0.1	No
2	0	down	32	-0.1	No
3	32	down	64	-0.1	No
4	64	right	72	-0.1	No
5	72	up	40	-0.1	No
6	40	up	8	-0.1	No
7	8	down	40	-0.1	No
8	40	up	8	-0.1	No
9	8	right	16	-0.1	No
10	16	up	16	-0.1	No
11	16	left	8	-0.1	No
12	8	left	0	-0.1	No
13	0	right	8	-0.1	No
14	8	left	0	-0.1	No
15	0	right	8	-0.1	No
16	8	down	40	-0.1	No
17	40	left	32	-0.1	No
18	32	right	40	-0.1	No
19	40	down	72	-0.1	No
20	72	up	40	-0.1	No
21	40	down	72	-0.1	No
22	72	left	64	-0.1	No
23	64	right	72	-0.1	No
24	72	left	64	-0.1	No
25	64	up	32	-0.1	No
26	32	right	40	-0.1	No
27	40	right	48	-0.1	No
28	48	left	40	-0.1	No
29	40	down	72	-0.1	No
30	72	right	80	-0.1	No
31	80	right	90	2.0	No
32	90	left	82	-0.1	No
33	82	right	90	-0.1	No
34	90	up	58	-0.1	No
35	58	up	27	1.0	No
36	27	up	27	-0.1	No
37	27	down	59	-0.1	No
38	59	up	27	-0.1	No
39	27	down	59	-0.1	No
40	59	down	91	-0.1	No

Stochastic trace

Step	State	Action	Next state	Reward	Done
1	0	left	0	-0.1	No
2	0	down	32	-0.1	No
3	32	down	64	-0.1	No
4	64	right	72	-0.1	No
5	72	up	40	-0.1	No
6	40	up	8	-0.1	No
7	8	down	0	-0.1	No
8	0	up	0	-0.1	No
9	0	right	8	-0.1	No
10	8	up	8	-0.1	No
11	8	left	0	-0.1	No
12	0	left	0	-0.1	No
13	0	right	8	-0.1	No
14	8	left	0	-0.1	No
15	0	right	8	-0.1	No
16	8	down	16	-0.1	No
17	16	left	8	-0.1	No
18	8	right	16	-0.1	No
19	16	down	48	-0.1	No
20	48	up	16	-0.1	No
21	16	down	48	-0.1	No
22	48	left	40	-0.1	No
23	40	right	48	-0.1	No
24	48	left	80	-0.1	No
25	80	up	48	-0.1	No
26	48	right	56	-0.1	No
27	56	right	25	1.0	No
28	25	left	17	-0.1	No
29	17	down	49	-0.1	No
30	49	right	57	-0.1	No
31	57	right	57	-0.1	No
32	57	left	49	-0.1	No
33	49	right	57	-0.1	No
34	57	up	25	-0.1	No
35	25	up	25	-0.1	No
36	25	up	17	-0.1	No
37	17	down	49	-0.1	No
38	49	up	17	-0.1	No
39	17	down	9	-0.1	No
40	9	down	41	-0.1	No